

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут» імені Ігоря Сікорського
Факультет інформатики та обчислювальної техніки
Кафедра автоматики та управління в технічних системах

Лабораторна робота № 1

із дисципліни: «Технології розроблення програмного забезпечення»
на тему: «Знайомство з Git»

Виконав:
студент групи ІА-34

Єресько Ілля

Перевірил:
Мягкий Михайло Юрійович

Короткі теоретичні відомості

1.2.1. Призначення систем керування версіями

Системи керування версіями (Version Control System або Source Control System) - це програмні інструменти, які допомагають командам розробників контролювати зміни у вихідному коді під час роботи над проектом [1].

Такі системи дають змогу зберігати зміни у файлах у репозиторії й створювати нові ревізії після кожної фіксації. Завдяки цьому можна повертатися до попередніх станів коду, аналізувати внесені зміни, знаходити причини помилок та визначати, хто і коли зробив той чи інший внесок.

Найчастіше системи контролю версій використовують у розробці програмного забезпечення для збереження вихідних кодів, що постійно змінюються. Проте вони корисні й в інших сферах, де працюють з великими обсягами документів, що регулярно оновлюються. Зокрема, такі системи застосовують у САПР, а також у складі PDM-систем. Вони є важливою частиною інструментів конфігураційного керування (Software Configuration Management Tools).

1.2.2. Історія розвитку систем контролю версій

Розвиток систем контролю версій умовно можна розділити на чотири основні етапи:

первинний, централізований, децентралізований та етап хмарних платформ.

Ранній етап

На початковому етапі робота з версіями обмежувалась локальним керуванням окремими файлами.

Найпростішою «системою контролю» було копіювання проекту з новою назвою (наприклад, *project_1*, *project_new*, *project_latest*), зазвичай у вигляді zip-, arj- або tar-архівів. Такий підхід, очевидно, не був повноцінною системою контролю версій.

Першою реальною системою стала **RCS**, представлена у 1982 році.

RCS

RCS запровадила зберігання змін у вигляді «дельт» тобто зберігалися лише змінені рядки, а не весь файл.

Водночас система мала низку обмежень:

- працювала лише з текстовими файлами;
- не мала центрального репозиторію кожен файл мав окремий *.rcs* репозиторій;
- при великій кількості файлів створювалась така ж кількість службових rcs-файлів;
- неможливість повноцінної роботи з версіями та гілками.

Етап централізованих систем

На початку 1990-х починається період централізованих систем контролю версій. З'являються рішення, які дозволяли кільком розробникам одночасно працювати через сервер.

Однією з найпопулярніших того часу була **CVS**, що значно змінила уявлення про спільну роботу та централізовані репозиторії.

SVN

SVN став логічним продовженням і покращенням CVS. Його особливості:

- надійна та швидка робота;
- проста модель використання (commit/update);
- покращене керування комітами та резервним копіюванням;
- клієнт-серверна архітектура.

Попри переваги, SVN мав труднощі зі створенням і злиттям гілок, а також погано опрацьовував конфлікти. Проте й сьогодні його все ще застосовують у ряді проєктів.

Етап децентралізації

Децентралізовані системи усунули залежність від центрального сервера: кожен розробник отримав повну копію репозиторію.

У 1992 році з'явилася одна з найпотужніших систем - **ClearCase**, яка запропонувала:

- віртуальну файлову систему,
- розширені можливості аудиту та злиття змін,
- сумісність з багатьма платформами.

Подальший прорив стався у 2005 році - створено **Git** та **Mercurial**. Обидві системи запропонували високу швидкість, надійність і гнучкість, хоча реалізовані вони різними способами. Після 2018 року Git остаточно став домінуючою безкоштовною системою контролю версій.

Git

Git був розроблений Лінусом Торвальдсом, щоб забезпечити розробників Linux незалежною системою контролю версій. Git працює за принципом розподіленої моделі: кожен користувач має власний повний репозиторій, а синхронізація з центральним сервером виконується лише за потреби .

Система автоматизує складні операції з гілкуванням та об'єднанням і зберігає зміни у вигляді наборів змін (changesets), кожен із яких має унікальний хеш.

Mercurial

Mercurial був створений після відмови BitKeeper від безкоштовної моделі. Він багато в чому схожий на Git, але використовує інший підхід до зберігання даних - ревлоги (revlog), що представляють собою плоску структуру файлів і каталогів.

Етап хмарних платформ

З приблизно 2010 року починається новий етап розвитку - інтеграція систем контролю версій із хмарними платформами та сервісами автоматизації.

Найпопулярніші хмарні платформи, що базуються на Git:

- GitHub
- GitLab
- Bitbucket

Вони інтегрують:

- CI/CD,
- спільну роботу,
- DevOps-інструменти,
- автоматичне тестування,
- аналітику.

Суть цього етапу - поєднання контролю версій із хмарною інфраструктурою для глобальної взаємодії та автоматизації процесів.

1.2.3. Робота з Git

Git дозволяє працювати як через командний рядок, так і за допомогою графічних клієнтів.

Командний рядок дає повний доступ до можливостей Git і дозволяє створювати складні комбінації команд.

Графічні інструменти роблять репозиторій наочним, хоч і зазвичай не покривають усіх функцій Git.

Приклади GUI-клієнтів: Git Extensions, SourceTree, GitKraken, GitHub Desktop.

Основні команди Git :

1.git clone - завантаження копії віддаленого репозиторію.

2. **git fetch / git pull** - отримання змін із віддаленого репозиторію.
3. **git commit** - фіксація зроблених змін у локальному репозиторії.
4. **git push** - передача локальних змін у віддалений репозиторій.
5. **update** - оновлення до певної версії з репозиторію.
6. **git merge** - злиття гілки з поточною гілкою.

Мета: Навчитися виконувати основні операції в роботі з децентралізованими системами контролю версій на прикладі роботи з сучасною системою Git.

```
C:\Users\eresk>git init 111
Initialized empty Git repository in C:/Users/eresk/111/.git/

C:\Users\eresk>cd 111

C:\Users\eresk\111>git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)

C:\Users\eresk\111>echo "t" > t.txt

C:\Users\eresk\111>git add t.txt

C:\Users\eresk\111>git commit -m "tt"
[master (root-commit) aa17e74] tt
 1 file changed, 1 insertion(+)
 create mode 100644 t.txt
```

Створюємо порожній репозиторій, додаємо текстовий файл, робимо коміт

```
C:\Users\eresk\111>echo "t" > t.txt

C:\Users\eresk\111>git add t.txt

C:\Users\eresk\111>git commit -m "tt"
[master (root-commit) aa17e74] tt
 1 file changed, 1 insertion(+)
 create mode 100644 t.txt

C:\Users\eresk\111>git branch b1

C:\Users\eresk\111>git switch -c b2
Switched to a new branch 'b2'

C:\Users\eresk\111>git branch
  b1
* b2
  master
```

Двома способами створюємо гілки в нашому репозиторії

```
C:\Users\eresk\111>echo "hh" > t.txt

C:\Users\eresk\111>git add t.txt

C:\Users\eresk\111>git commit -m "rr"
[b2 ae0a3d0] rr
1 file changed, 1 insertion(+), 1 deletion(-)

C:\Users\eresk\111>git checkout b1
Switched to branch 'b1'

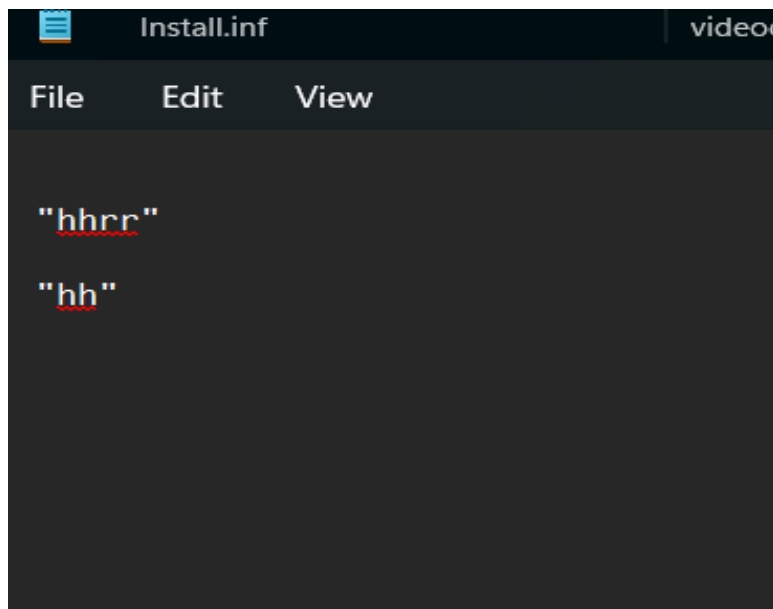
C:\Users\eresk\111>echo "hhrr" > t.txt

C:\Users\eresk\111>git add t.txt

C:\Users\eresk\111>git commit -m "rrr"
[b1 9f6a741] rrr
1 file changed, 1 insertion(+), 1 deletion(-)

C:\Users\eresk\111>git merge b2
Auto-merging t.txt
CONFLICT (content): Merge conflict in t.txt
Automatic merge failed; fix conflicts and then commit the result.
```

Створюємо конфлікт змінюючи файл з двох різних гілок



Вирішуємо конфлікт змінюючи файл вручну


```

C:\Users\eresk\111>git log --graph --all
*   commit 3772e190ee10ed147539ed501947ee8af744953e (HEAD -> b1)
| \
|  Merge: 9f6a741 ae0a3d0
|  Author: Ilya Yeresko <ereskoilal@gmail.com>
|  Date:   Sat Oct 4 14:39:07 2025 +0300
|
|      'ssa'
|
| *   commit ae0a3d0aef795bb0dddf8d4c349e6973bd862be9 (b2)
|  |
|  | Author: Ilya Yeresko <ereskoilal@gmail.com>
|  | Date:   Sat Oct 4 14:35:08 2025 +0300
|  |
|  |      rr
|  |
|  *   commit 9f6a74181994a950fd9943ee608ae83a49920a
|  | /
|  | Author: Ilya Yeresko <ereskoilal@gmail.com>
|  | Date:   Sat Oct 4 14:35:20 2025 +0300
|  |
|  |      rrr
|  |
|  *   commit aa17e74e3791ed0dcfd93275d24a630baad06afc (master)
|  |
|  | Author: Ilya Yeresko <ereskoilal@gmail.com>
|  | Date:   Sat Oct 4 14:34:50 2025 +0300
|  |
|  |      tt

```

Перевіряємо

Висновок: Я навчився виконувати основні операції git

Висновок

Системи контролю версій є невід'ємною частиною сучасної розробки програмного забезпечення, оскільки забезпечують керування змінами коду, збереження історії розробки та синхронізацію роботи між учасниками команди. Розвиток таких систем пройшов кілька етапів - від простих локальних рішень до централізованих платформ і децентралізованих систем, серед яких Git посідає провідне місце.

Git, як розподілена система контролю версій, надає розробникам повну копію репозиторію, підтримує гнучку роботу з гілками, забезпечує високу швидкість та надійність фіксації змін. Його використання значно спрощує командну взаємодію, дозволяє

ефективно контролювати історію змін та інтегрується з сучасними хмарними платформами.

У ході вивчення теми було опановано основні операції роботи з Git - клонування репозиторію, створення комітів, синхронізація змін, злиття гілок та взаємодія з віддаленими репозиторіями. Це формує базові навички роботи з найпоширенішою сучасною системою контролю версій та закладає основу для подальшого ефективного використання інструментів командної розробки.

Контрольні запитання

1. Що таке система контролю версій (СКВ)?

Система контролю версій - це програмний інструмент, який дозволяє відстежувати, зберігати та керувати змінами у файлах, найчастіше у вихідному коді програм. Вона фіксує кожну зміну у вигляді окремої версії, зберігає історію редагувань та дає можливість повернутися до будь-якого стану проєкту. СКВ дозволяє декільком розробникам працювати над одним проєктом одночасно, уникати втрати даних, порівнювати версії, аналізувати причини помилок та організовувати структуру проєкту. Таким чином, система контролю версій є основою командної розробки та важливим засобом забезпечення надійності програмного продукту.

2. Відмінності між розподіленою та централізованою СКВ

Централізована система контролю версій має один центральний сервер, де зберігається повний репозиторій, а користувачі отримують лише робочі копії файлів. Для роботи з історією, комітами чи створенням гілок потрібен постійний доступ до сервера. У розподіленій системі кожен розробник має повну копію репозиторію разом із історією, що дозволяє виконувати коміти, переглядати історію та створювати гілки навіть офлайн. Децентралізовані СКВ забезпечують більшу надійність, оскільки кожен локальний репозиторій слугує резервною копією, та пропонують значно гнучкішу роботу з гілками і синхронізацією.

3. Різниця між stage та commit в Git

Stage (або *staging area*) - це проміжна зона, куди додаються файли перед створенням коміту. На цьому етапі розробник визначає, які саме зміни мають увійти у наступну версію. Commit — це вже зафіксована версія коду, що зберігається в історії репозиторію. Таким чином, staging дозволяє вибрати

конкретні зміни для фіксації, а `commit` створює остаточний запис у журналі змін. Це розділення на два етапи дає змогу формувати логічно зрозумілі коміти та уникати випадкового додавання зайвих змін.

4. Як створити гілку в Git?

Гілка створюється командою `git branch <назва-гілки>`, яка створює нове посилання на поточний коміт. Після цього для переходу в гілку використовують `git checkout <назва-гілки>` або сучасну команду `git switch <назва-гілки>`. Зазвичай створення гілки супроводжується переходом у неї, щоб почати роботу з новими змінами, ізолювавши їх від основної гілки (наприклад, `main` чи `master`).

5. Як створити або скопіювати репозиторій Git з віддаленого серверу?

Для отримання копії віддаленого репозиторію використовується команда `git clone <url>`. Вона створює локальну директорію, завантажує весь репозиторій разом із гілками, історією та налаштовує зв'язок із віддаленим сервером. Після клонування користувач може одразу працювати з проєктом та синхронізувати зміни з сервером.

6. Що таке конфлікт злиття, як створити та як вирішити?

Конфлікт злиття виникає тоді, коли дві гілки містять зміни в одних і тих самих рядках файлу, і Git не може самостійно визначити правильний варіант. Створити конфлікт легко: достатньо у двох різних гілках змінити один і той самий рядок коду по-різному, а потім виконати `git merge`. Для вирішення конфлікту необхідно вручну відредагувати файл, залишивши потрібний варіант змін, після чого позначити його як вирішений (`git add`) та завершити злиття (`git commit`). Більшість IDE та інструментів Git також пропонують зручні графічні інтерфейси для розв'язання конфліктів.

7. Коли використовуються `merge`, `rebase`, `cherry-pick`?

Команда **`merge`** застосовується для об'єднання двох гілок в одну, зберігаючи історію змін обох гілок. **`Rebase`** перезаписує історію, "переносячи" коміти однієї гілки поверх іншої, що робить історію лінійною та чистою. Команда **`cherry-pick`** дозволяє взяти один конкретний коміт із будь-якої гілки та додати його в поточну гілку — це корисно, коли потрібна лише частина змін, а не вся гілка.

8. Як переглянути історію змін у консолі?

Для перегляду історії використовується команда `git log`. Вона показує коміти, авторів, дати та повідомлення. Існують розширені варіанти, наприклад `git log --oneline --graph --all`, що дозволяє переглядати дерево гілок у компактному форматі.

9. Як створити гілку в Git, не використовуючи git branch?

Гілку можна створити та одразу перейти в неї командою `git checkout -b <назва-гілки>` або сучасним варіантом `git switch -c <назва-гілки>`. Обидві команди створюють нову гілку та одразу активують її. Це зручно, коли потрібно швидко почати роботу.

10. Як підготувати всі зміни в поточній папці до коміту?

Для додавання всіх змін у поточній директорії та вкладених папках використовують команду `git add ..`. Вона додає усі нові, змінені та видалені файли до staging area, щоб вони увійшли в наступний коміт.

11. Як підготувати всі зміни в дочірній папці до коміту?

Щоб додати зміни лише в конкретній підпапці, використовують `git add <шлях до папки>`. Наприклад: `git add src/` — додасть усі зміни лише в директорії `src` і не зачепить інші папки.

12. Як переглянути список гілок у репозиторії?

Перелік локальних гілок можна переглянути командою `git branch`. Щоб побачити також віддалені гілки, використовують `git branch -a` або `git branch -r`. Це дозволяє швидко перевірити структуру гілкування в проєкті.

13. Як видалити гілку?

Гілку можна видалити командою `git branch -d <назва-гілки>` — ця команда видалить гілку лише якщо її зміни вже злиті. Якщо потрібно примусово видалити гілку, використовують `git branch -D <назва-гілки>`. Для віддаленої гілки застосовується `git push origin --delete <назва-гілки>`.

14. Які є способи створення гілки та в чому різниця?

Гілку можна створити двома основними способами:

1. `git branch <назва>` - створення гілки без автоматичного переходу в неї.
2. `git checkout -b <назва>` або `git switch -c <назва>` = створення гілки з одразу активним переходом.

Різниця полягає у зручності: перший спосіб використовується, коли потрібно просто створити кілька гілок, а другий — коли важливо одразу продовжити роботу саме в новій гілці.